

Trial_1: CNN architecture transformed into a **Multi-Scale Convolutional Neural Network (MSCNN)** using *parallel convolution layers* with different kernel sizes.

```
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
#from tensorflow.keras.models import Sequential
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Conv1D, BatchNormalization, Dropout, Concatenate, Flatten, Dense
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, roc_curve
```

load & Split data

↳ 2 cells hidden

Multi-Scale Convolutional Neural Network (MSCNN)

```
input_layer = Input(shape=(256, 1))

# Multi-scale convolution block
conv3 = Conv1D(filters=16, kernel_size=3, padding='same', activation='relu')(input_layer)
conv5 = Conv1D(filters=16, kernel_size=5, padding='same', activation='relu')(input_layer)
conv7 = Conv1D(filters=16, kernel_size=7, padding='same', activation='relu')(input_layer)

# Merge multi-scale features
merged = Concatenate(axis=-1)([conv3, conv5, conv7])
merged = BatchNormalization()(merged)
merged = Dropout(0.5)(merged)

# Additional convolution + dropout layers
conv = Conv1D(filters=32, kernel_size=3, activation='relu', padding='same')(merged)
conv = BatchNormalization()(conv)
conv = Dropout(0.5)(conv)

conv = Conv1D(filters=16, kernel_size=3, activation='relu', padding='same')(conv)
```

```
# Fully connected layers
flattened = Flatten()(conv)
dense = Dense(128, activation='relu')(flattened)
dense = Dense(128, activation='relu')(dense)
dense = Dense(128, activation='relu')(dense)
output = Dense(1, activation='sigmoid')(dense) # For binary classification
```

```
# Create and compile model
model = Model(inputs=input_layer, outputs=output)
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

```
# Train model
history = model.fit(Xtrain_resaped, Ytrain, epochs=60, batch_size=64, validation_split=0.2)
```

Show hidden output

```
# Evaluate the model on the test set
loss, accuracy = model.evaluate(Xtest_resaped, Ytest)
print(f"Test Accuracy: {accuracy:.2%}")
print(f"Test Loss: {loss:.4f}")

#-----
Ypred = model.predict(Xtest_resaped)
Ypred_classes = (Ypred > 0.5).astype(int)
Ypred_classes_flat = Ypred_classes.flatten()

# Print classification report (precision, recall, F1-score)
print("\nClassification Report:")
print(classification_report(Ytest, Ypred_classes_flat)) # Use Ytest directly
```

```
#-----
# Plot accuracy
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

# Plot loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.show()

#-----
roc_auc = roc_auc_score(Ytest, Ypred[:, 0])
print(f"\nROC AUC: {roc_auc:.4f}")

# Plot ROC curve
# Use Ytest for true labels and Ypred[:, 0] for predicted probabilities of the positive class
fpr, tpr, thresholds = roc_curve(Ytest, Ypred[:, 0])
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (area = %0.2f)' % roc_auc)
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.0])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc='lower right')
plt.show()

#-----
cm = confusion_matrix(Ytest, Ypred_classes_flat)
plt.figure(figsize=(8, 6))
class_labels = ['Class 0', 'Class 1'] # Or provide your actual class names if available
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_labels,
            yticklabels=class_labels)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()
```

